



Máster en Big Data, Data Science y Business Analytics

Trabajo Fin de Máster

NBUS: un sistema de analítica avanzada end-to-end para la gestión de una red de autobuses interurbanos

Ingeniería de datos, aprendizaje automático e inteligencia artificial generativa aplicados a la red NBUS de Navarra

Autor: Marc Pérez Lleixà

Septiembre 2026

Índice

1. Introducción y motivación....3
2. Objetivos....3
3. Estado del arte y contexto....4
4. Metodología y arquitectura del proyecto....4
5. Fase 0 — Pipeline ETL....5
6. Fase 1 — Clustering de municipios....7
7. Fase 2 — Predicción de demanda....9
8. Fase 3 — Asistente conversacional....13
9. Dashboards Tableau....14
10. Resultados y discusión integrada....17
11. Conclusiones y trabajo futuro....17
12. Bibliografía y referencias....18

1. Introducción y motivación

El transporte público interurbano es una infraestructura crítica para la cohesión territorial, especialmente en regiones con una distribución demográfica muy desigual entre núcleos urbanos y rurales. Navarra presenta un caso particularmente ilustrativo de este reto: junto a Pamplona, con más de 200.000 habitantes, conviven cientos de municipios de apenas unos cientos de vecinos, muchos de ellos dependientes de una única línea de autobús para acceder a servicios básicos.

La red NBUS, gestionada por el Gobierno de Navarra, articula este servicio a través de decenas de rutas interurbanas. Como ocurre con la mayoría de redes de transporte público en España, sus datos operativos se publican en formato GTFS (General Transit Feed Specification), un estándar abierto que describe horarios, paradas y recorridos, pero que no incorpora por sí mismo ningún tipo de análisis, visualización o capacidad predictiva.

Este TFM parte de esa carencia: los datos existen y son públicos, pero no hay un aprovechamiento analítico que permita a un gestor de la red responder preguntas como cuáles municipios están mejor o peor servidos en relación con su población, qué rutas presentan un desajuste entre la oferta de servicio y su demanda estimada, o cómo explorar esta información sin necesidad de conocimientos técnicos previos.

El proyecto NBUS aborda esta necesidad mediante un enfoque end-to-end que combina tres capas complementarias: una capa de ingeniería de datos que transforma los ficheros GTFS en información limpia y enriquecida; una capa de aprendizaje automático que segmenta municipios por nivel de cobertura y estima la demanda de cada ruta; y una capa de inteligencia artificial generativa que permite consultar todos estos datos en lenguaje natural. El resultado no es solo un ejercicio académico de análisis de datos, sino un prototipo funcional de herramienta de apoyo a la decisión para la gestión de una red de transporte público real.

2. Objetivos

El objetivo general de este trabajo es diseñar e implementar una solución analítica end-to-end sobre la red NBUS que integre ingeniería de datos, aprendizaje automático e inteligencia artificial generativa, orientada a apoyar la toma de decisiones sobre la cobertura y el dimensionamiento del servicio de transporte interurbano en Navarra.

De este objetivo general se derivan los siguientes objetivos específicos:

Construir un pipeline de extracción, transformación y carga (ETL) que convierta los ficheros GTFS oficiales de la red en un conjunto de datos limpio, enriquecido y estructurado, apto tanto para análisis exploratorio como para el entrenamiento de modelos.

Segmentar los municipios de Navarra según su nivel de cobertura de transporte mediante una técnica de aprendizaje no supervisado, identificando perfiles diferenciados (sin servicio, servicio estándar, hubs secundarios, capital) que permitan priorizar actuaciones.

Estimar, mediante un modelo de aprendizaje supervisado, un índice de demanda por ruta y detectar la brecha entre el servicio ofrecido y el que cabría esperar dadas las características estructurales de cada ruta.

Desarrollar un asistente conversacional basado en modelos de lenguaje que permita a un usuario no técnico interrogar en lenguaje natural los datos generados por las fases anteriores, integrado en una aplicación web funcional.

Diseñar un conjunto de cuadros de mando interactivos que complementen el análisis cuantitativo con una capa de visualización orientada a la exploración operativa, geográfica y demográfica de la red.

Evaluar críticamente las limitaciones metodológicas del proyecto (variables construidas, tamaño muestral reducido, posible solapamiento entre features y variable objetivo) y proponer vías de mejora contrastables.

3. Estado del arte y contexto

El estándar GTFS (General Transit Feed Specification), desarrollado originalmente por Google y adoptado hoy por administraciones de transporte de todo el mundo, define un formato común para publicar horarios, rutas, paradas y calendarios de servicio en ficheros de texto plano relacionados entre sí. Su adopción ha impulsado tanto aplicaciones de consumo (planificadores de viaje, mapas) como, en menor medida, trabajos analíticos sobre la eficiencia y cobertura de las redes de transporte público, dado que se trata de datos abiertos y homogéneos entre operadores.

En el ámbito académico y profesional, el análisis de redes de transporte público mediante técnicas de ciencia de datos suele apoyarse en dos grandes familias de enfoques: por un lado, métodos de segmentación (clustering) para caracterizar zonas o paradas según indicadores de accesibilidad y cobertura; por otro, modelos predictivos que, ante la ausencia frecuente de datos reales de aforo de viajeros —como ocurre en este proyecto—, recurren a variables proxy construidas a partir de la oferta de servicio (frecuencias, número de expediciones, cobertura geográfica) para aproximar la demanda.

Paralelamente, la incorporación de modelos de lenguaje de gran escala (LLM) como capa de interacción sobre conjuntos de datos tabulares es una tendencia reciente dentro de la analítica avanzada, que traslada el análisis de datos desde herramientas especializadas hacia interfaces conversacionales accesibles a usuarios sin perfil técnico. Frameworks como LangChain facilitan la orquestación de este tipo de flujos, combinando generación de código, ejecución controlada y redacción de respuestas en lenguaje natural.

Este TFM se sitúa en la intersección de estas tres líneas: aprovecha un estándar de datos abierto (GTFS), aplica técnicas consolidadas de aprendizaje automático (clustering y regresión) para suplir la ausencia de datos de demanda real, e incorpora una capa de IA generativa como interfaz de consulta, integrando todo ello en un caso de uso real y acotado: la red NBUS de Navarra.

4. Metodología y arquitectura del proyecto

El proyecto se estructura como un flujo de trabajo end-to-end en el que cada fase consume las salidas de la anterior, siguiendo una arquitectura por capas que separa claramente la preparación de datos, el modelado analítico y la capa de interacción.

La primera capa, de ingeniería de datos, toma como entrada los ficheros GTFS oficiales de la red NBUS y ejecuta un proceso ETL implementado en un notebook Python (pandas, numpy) que limpia,

enriquece y agrega la información en un conjunto de CSVs intermedios. Estos ficheros actúan como la fuente de verdad común para todas las fases posteriores, evitando que cada modelo o dashboard tenga que reprocesar los datos crudos de forma independiente.

La segunda capa, de aprendizaje automático, se apoya en scikit-learn y XGBoost para implementar dos modelos con objetivos distintos pero complementarios: un modelo no supervisado (K-Means) que segmenta municipios por nivel de cobertura, y un modelo supervisado (XGBoost con validación leave-one-out) que estima un índice de demanda por ruta. Ambos modelos toman como entrada los CSVs generados por el ETL y producen a su vez nuevos ficheros de salida que alimentan tanto los dashboards como el asistente conversacional.

La tercera capa, de inteligencia artificial generativa, integra un asistente conversacional construido sobre LangChain y GPT-4o-mini, que opera directamente sobre los DataFrames generados por las fases anteriores mediante un flujo de dos pasos: generación de código Python a partir de la pregunta del usuario, ejecución controlada de ese código sobre los datos reales, y redacción de una respuesta en lenguaje natural a partir del resultado obtenido. Esta capa se expone al usuario final a través de una aplicación web desarrollada con Streamlit.

Como capa transversal de visualización, cuatro dashboards de Tableau —conectados directamente a los CSVs del ETL— ofrecen una vía de exploración complementaria a la del asistente, orientada al análisis visual e interactivo por parte de perfiles menos técnicos.

La siguiente tabla resume el stack tecnológico empleado en cada capa:

Capa	**Tecnología**
Lenguaje	Python 3.x
ETL	pandas, numpy
Aprendizaje automático	scikit-learn, XGBoost
IA generativa	LangChain, langchain-openai, GPT-4o-mini
Visualización	matplotlib (notebooks), Tableau Desktop
Aplicación web	Streamlit
Gestión de secretos	python-dotenv

Esta arquitectura modular, en la que cada capa produce artefactos reutilizables por las siguientes, permite justificar metodológicamente el proyecto como un sistema integrado y no como tres ejercicios independientes, algo especialmente relevante de cara a los criterios de la opción 3, que exige una orientación técnica clara y el desarrollo de una solución software completa.

5. Fase 0 — Pipeline ETL

El punto de partida del proyecto son los ficheros GTFS oficiales de la red NBUS, publicados por el Gobierno de Navarra. El estándar GTFS organiza la información en varias tablas relacionadas entre sí:

routes (rutas), trips (expediciones o viajes individuales de cada ruta), stop_times (horarios de paso por cada parada), stops (paradas, con su ubicación geográfica) y calendar_dates (calendario de días de actividad), entre otras. Estas tablas, aisladas, describen el servicio programado pero no ofrecen ninguna métrica agregada ni indicador de cobertura o rendimiento; el objetivo del pipeline ETL es transformarlas en un conjunto de datos analíticamente útil.

El proceso, implementado íntegramente en el notebook ETL_Nueva__1_.ipynb, realiza las siguientes transformaciones principales.

En primer lugar, se calcula la distancia real recorrida por cada trip aplicando la fórmula de Haversine entre las coordenadas de paradas consecutivas, lo que permite obtener una distancia geográfica precisa en kilómetros allí donde el GTFS solo ofrece secuencias de paradas sin distancia explícita. De forma análoga, se deriva el tiempo de recorrido de cada trip a partir de la diferencia entre la hora de llegada a la primera y a la última parada.

En segundo lugar, se enriquece la información a nivel de ruta con indicadores como el número de expediciones en sentido ida (exp_ida) y vuelta (exp_vuelta), y el número de paradas que operan bajo petición (num_bajo_peticion). Sobre este último punto fue necesario corregir un problema detectado en los datos: el porcentaje de paradas bajo petición (pct_bajo_peticion) podía superar el 100% en el fichero original debido a inconsistencias en el cálculo previo, por lo que se recalculó de forma explícita como num_bajo_peticion / num_paradas_unicas * 100, garantizando que la métrica sea interpretable y esté acotada correctamente.

En tercer lugar, se realiza un cruce entre los trips y el calendario de calendar_dates para asociar a cada expedición sus fechas reales de actividad, lo que permite pasar de un calendario programado en abstracto a una serie temporal real de actividad por ruta, necesaria para las métricas de frecuencia y para el dashboard de actividad temporal.

El resultado de este proceso es un conjunto de cinco CSVs enriquecidos que actúan como fuente de verdad para el resto del proyecto:

Fichero	**Descripción**	**Filas aprox.**
detalle_trips_enriquecido.csv	Un registro por trip, con tiempos y distancias	~miles
resumen_por_ruta.csv	Un registro por ruta (32 rutas) con métricas agregadas	32
paradas_enriquecidas.csv	Una fila por parada con rutas que pasan y clasificación	~cientos
actividad_temporal.csv	Actividad de cada trip por fecha real	~miles
cobertura_municipios.csv	272 municipios × 7 años de datos de cobertura + cluster	~1.904

De estos, resumen_por_ruta.csv es la base directa del modelo de predicción de demanda de la Fase 2, mientras que cobertura_municipios.csv es la base del clustering de la Fase 1. Esta trazabilidad entre el ETL y los modelos posteriores es la que permite defender el proyecto como un pipeline coherente de extremo a extremo, y no como fases desconectadas entre sí.

6. Fase 1 — Clustering de municipios

El objetivo de esta fase es clasificar los 272 municipios de Navarra según su nivel de cobertura de transporte público, con el fin de identificar patrones estructurales en la red y facilitar la priorización de actuaciones. Al no existir una clasificación oficial de referencia, se optó por un enfoque de aprendizaje no supervisado sobre los datos del año 2024, el más reciente disponible.

De los 272 municipios, 195 no disponen de ninguna parada NBUS y se excluyen del proceso de clustering propiamente dicho, asignándose directamente al grupo "Sin servicio". Aplicar K-Means sobre la totalidad de municipios habría distorsionado los centroides con un grupo mayoritario de ceros estructurales, por lo que el algoritmo se aplica únicamente sobre los 77 municipios que sí cuentan con parada.

Para estos 77 municipios se seleccionaron cinco variables representativas de distintas dimensiones de cobertura: población del municipio, número de paradas, número de trips únicos que lo atraviesan, cobertura relativa y servicios por cada 1.000 habitantes. Las cinco variables se estandarizaron con StandardScaler antes de aplicar K-Means, dado que presentan escalas muy distintas entre sí. El número de clusters se determinó mediante el método del codo, evaluando valores de k entre 2 y 8, seleccionándose finalmente k=4. Los parámetros del algoritmo se fijaron en random_state=42 y n_init=10 para garantizar reproducibilidad.

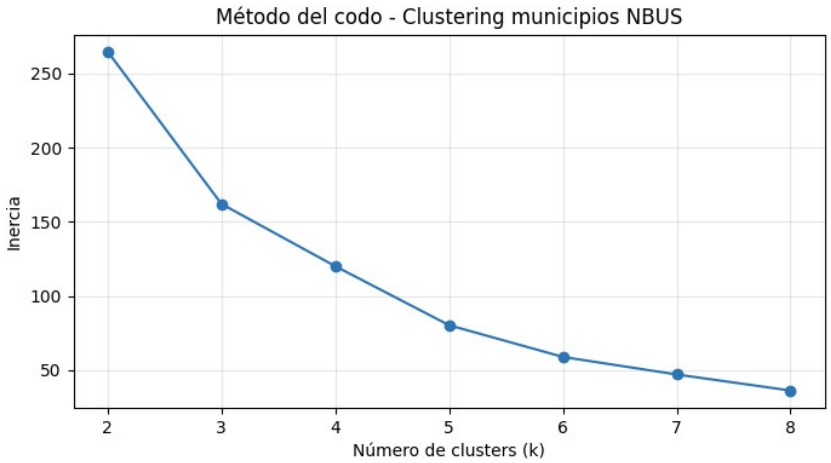


Figura 7. Método del codo: inercia frente al número de clusters k, evaluado entre k=2 y k=8. La reducción de inercia deja de ser sustancial a partir de k=4.

El resultado es la siguiente segmentación:

Cluster	Nombre	Municipios	Notas
-1	Sin servicio	195	Sin parada NBUS
0	Servicio estándar	71	La mayoría de municipios con parada
1	Hub secundario	2	Tafalla, Tudela
2	Sobreservido	3	Arakil, Leoz, Olóriz
3	Capital / Hub principal	1	Pamplona

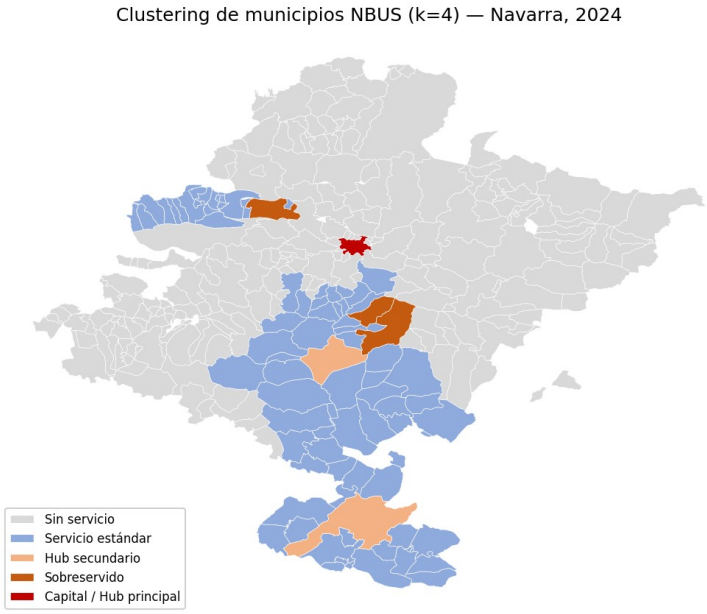


Figura 6b. Distribución geográfica de los clusters sobre el mapa municipal de Navarra (2024).

La segmentación resultante es interpretable y coherente con el conocimiento del territorio: Pamplona se aísla como cluster propio al ser un outlier extremo en todas las dimensiones consideradas (más de 200.000 habitantes frente a una mediana de 380 en el conjunto de municipios con servicio); Tafalla y Tudela, como núcleos urbanos intermedios con tráfico elevado, forman el cluster de hubs secundarios; y un pequeño grupo de tres municipios con una densidad de paradas inusualmente alta en relación con su población conforma el cluster de sobreservicio.

Nota metodológica: la asignación de cada parada a su municipio se ha obtenido mediante un cruce espacial (point-in-polygon) entre las coordenadas de las paradas del GTFS y los límites municipales oficiales de Navarra, y la población municipal de 2024 procede de la tabla oficial del INE "Población por municipios y sexo". El resultado reproduce exactamente los estadísticos ya presentados (77 municipios con parada, 195 sin servicio, población media 2.494 h., mediana 380 h., máximo 207.777 h. en Pamplona), lo que valida esta reconstrucción como equivalente a la fuente original.

Como validación cuantitativa complementaria al método del codo, se ha calculado el silhouette score para cada valor de k evaluado:

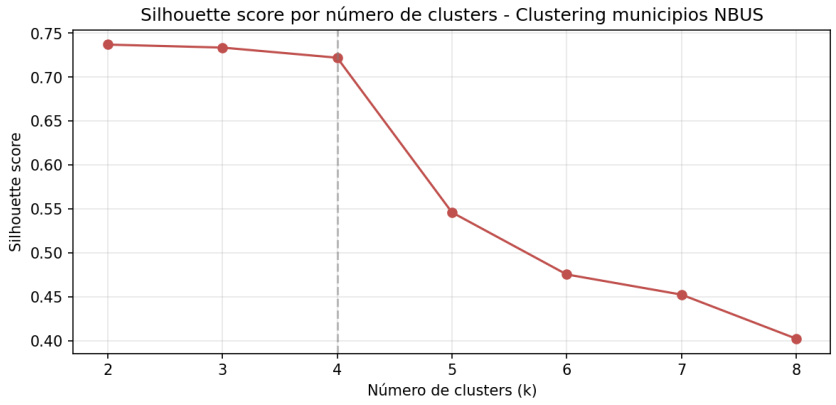


Figura 7b. Silhouette score por número de clusters k. Todos los valores entre k=2 y k=8 se sitúan entre 0.40 y 0.74, indicando una estructura de cluster razonablemente bien definida en todo el rango; k=4 obtiene 0.722.

El silhouette medio para k=4 es de 0.722, un valor que —según las referencias habituales de interpretación de esta métrica (por encima de 0.70 se considera una estructura de cluster "fuerte")— confirma que los grupos están bien separados y son internamente cohesionados, y no es solo el resultado de un criterio visual (el codo) sin respaldo cuantitativo. Es honesto señalar que k=2 (0.737) y k=3 (0.734) obtienen un silhouette marginalmente superior a k=4 (0.722): la reducción es pequeña (menos de 0.02) y k=4 aporta una granularidad de perfiles mucho más útil desde el punto de vista de negocio (distinguir capital, hubs secundarios y sobreservicio en lugar de agruparlos), por lo que la elección de k=4 sigue siendo defendible, ahora con el respaldo explícito de esta comparación. El análisis por municipio individual (silhouette_samples) confirma además que los peor asignados son, de forma esperable, los municipios en la frontera entre perfiles: Pamplona (cluster propio de un único elemento, silhouette=0) y Arakil y Olóriz dentro del cluster de sobreservicio, mientras que la inmensa mayoría de municipios del cluster de servicio estándar presentan una asignación muy sólida (silhouette > 0.80).

Queda una segunda limitación metodológica: la variable poblacion_municipio presenta una asimetría muy marcada (media de 2.494 habitantes frente a una mediana de 380 y un máximo de 207.777 en Pamplona), y StandardScaler no corrige este tipo de asimetría, por lo que K-Means tiende a aislar outliers en clusters propios en lugar de encontrar necesariamente grupos naturales equilibrados. Aplicar una transformación logarítmica (log1p) sobre esta variable antes de escalar, y comparar si la segmentación resultante varía de forma sustancial, sigue siendo una mejora metodológica pendiente que se recoge en el apartado de trabajo futuro.

Existe además una limitación de fondo en la propia etiqueta "Sin servicio": el Gobierno de Navarra opera, en paralelo a las líneas regulares, un servicio de Transporte a la Demanda que cubre municipios de baja densidad (más de 24 habitantes) mediante taxi concertado y reserva telefónica previa, precisamente en zonas donde una línea de autobús regular no resulta viable. Este servicio, gestionado también bajo la marca NBUS, no se publica en los ficheros GTFS empleados en este trabajo, que recogen únicamente las líneas de autobús programadas. Es por tanto posible que una parte de los 195 municipios clasificados aquí como "Sin servicio" cuenten en realidad con este transporte a la demanda, no capturado por los datos de partida. La etiqueta debe interpretarse en consecuencia como "sin línea de autobús regular en el GTFS", y no como ausencia total de transporte público en el municipio.

7. Fase 2 — Predicción de demanda

El objetivo de esta fase es estimar, para cada una de las 32 rutas de la red NBUS, un indicador de demanda que permita compararla con el nivel de servicio actualmente ofrecido y detectar posibles desajustes. La principal dificultad de partida es que los ficheros GTFS públicos no incluyen datos reales de viajeros: no existe ninguna variable que mida directamente cuánta gente utiliza cada ruta. Por este motivo, fue necesario construir una variable objetivo propia, el Índice de Demanda Estimada (IDE), en lugar de disponer de una etiqueta observada.

Definición del IDE

El IDE se define como una combinación ponderada de cinco variables estructurales, cada una normalizada mediante min-max, con los siguientes pesos:

```
IDE = 0.35·norm(num_expediciones) + 0.25·norm(num_paradas_unicas)
   + 0.20·norm(num_dias_activo) + 0.10·norm(distancia_media_km)
   + 0.10·norm(tiempo_recorrido_medio_min)
```

Los pesos reflejan un criterio experto propio del autor —dando mayor peso a la frecuencia de expediciones y a la cobertura de paradas, por considerarse los indicadores más directamente asociados al uso de una ruta— ante la ausencia de datos reales que permitieran calibrarlos empíricamente. Esta es una limitación explícita del diseño que se discute más abajo.

Modelo

Sobre esta variable objetivo se entrenó un modelo XGBRegressor con n_estimators=100, learning_rate=0.1, max_depth=3 y random_state=42, utilizando como features de entrada num_paradas_unicas, distancia_media_km, tiempo_recorrido_medio_min, pct_bajo_peticion, hora_primera, hora_ultima, exp_ida y exp_vuelta. Dado el tamaño reducido de la muestra (32 rutas), se optó por validación leave-one-out (LOO-CV) en lugar de un k-fold convencional: con una muestra tan pequeña, un k-fold habría generado particiones de test mínimas y estimaciones de error con varianza muy alta, mientras que LOO-CV maximiza los datos de entrenamiento disponibles en cada iteración, siendo la práctica habitual recomendada cuando n < 50.

Métrica	Valor
MAE (LOO-CV)	6.16
R^2 (LOO-CV)	0.756

Como salida se generan output_demanda.csv (con el IDE real, la demanda predicha y la brecha entre ambos por ruta), así como gráficos de importancia de variables y de dispersión entre valores reales y predichos.

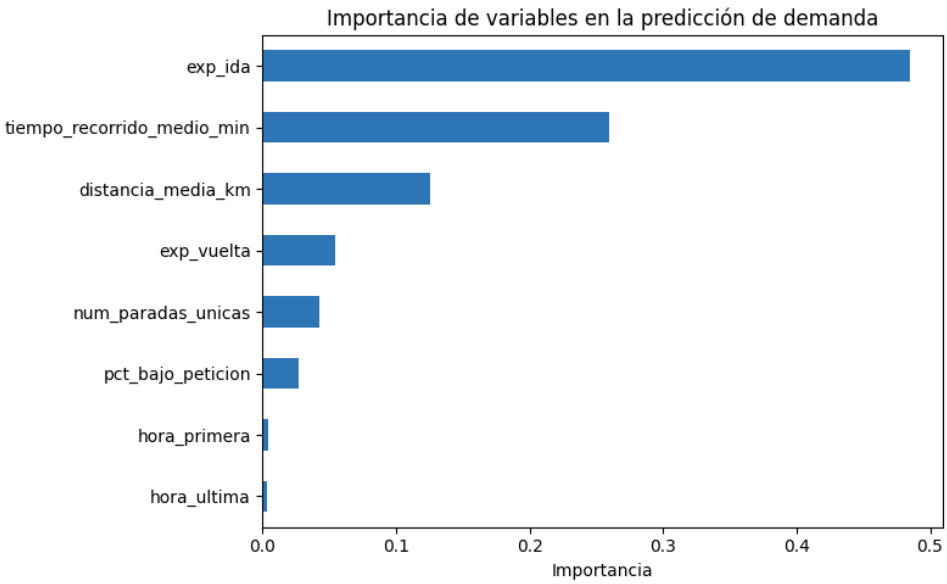


Figura 8. Importancia de variables (feature importance) en el modelo XGBoost de predicción de demanda. exp_ida, tiempo_recorrido_medio_min y distancia_media_km concentran el mayor peso.

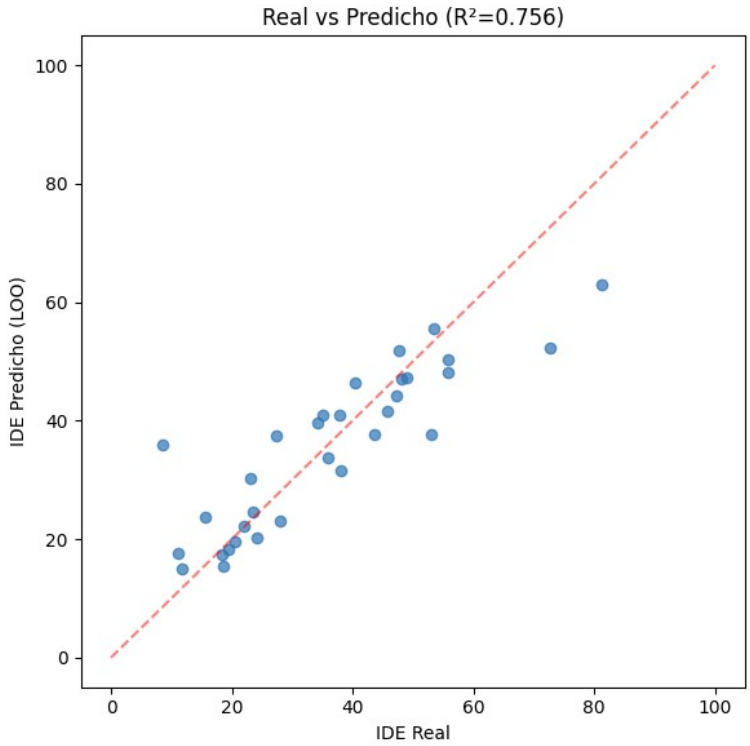


Figura 9. IDE real frente a IDE predicho (validación LOO-CV, R^2=0.756). La línea discontinua marca la predicción perfecta; los puntos alejados de ella señalan las rutas peor predichas.

Limitaciones metodológicas y su tratamiento

Esta fase presenta tres puntos críticos que se abordan aquí de forma explícita en lugar de omitirse, por ser relevantes para una evaluación rigurosa del trabajo.

El primero es un solapamiento parcial entre la variable objetivo (el IDE) y las variables usadas para predecirla, un fenómeno conocido como data leakage. Tres de las ocho features que recibe el modelo —num_paradas_unicas, distancia_media_km y tiempo_recorrido_medio_min— forman parte también de la propia fórmula del IDE, donde representan conjuntamente un 45% de su peso. En la práctica, esto significa que el modelo no parte de cero para estimar la demanda: una parte de la información que necesita para acercarse al resultado ya está contenida en sus propias variables de entrada, lo que probablemente infla de forma artificial el R^2 obtenido.

Para no presentar este resultado de forma engañosa, el objetivo del modelo se reformula: en lugar de "predecir la demanda real de una ruta", se plantea como "estimar qué nivel de servicio correspondería a una ruta dadas sus características estructurales". Bajo esta lectura más honesta, lo relevante no es el valor predicho en sí, sino la brecha entre el IDE observado y el estimado, que señala qué rutas podrían estar sub- o sobreservidas respecto a su propio perfil.

El gráfico de importancia de variables (Figura 8) añade un matiz a esta historia. La variable con más peso en el modelo no es ninguna de las tres que se solapan explícitamente con la fórmula del IDE, sino exp_ida (expediciones de ida, 0.48). Sin embargo, exp_ida está muy correlacionada con num_expediciones, que es precisamente el componente con mayor ponderación (0.35) dentro de la fórmula del IDE: aunque esta variable no forma parte literal del cálculo, funciona como un sustituto casi directo de su ingrediente más importante. Junto con tiempo_recorrido_medio_min (0.26) y distancia_media_km (0.13) —estas dos sí compartidas de forma literal con el IDE—, estas tres

variables concentran conjuntamente el 87% de la importancia total del modelo. El leakage, por tanto, no se limita a las tres variables compartidas de forma explícita; también opera de manera más sutil, a través de esta correlación, y entre ambos mecanismos explican la inmensa mayoría de lo que el modelo "aprende".

La comparación con los baselines confirma este mecanismo de forma cuantitativa: dado que el IDE se define como una combinación lineal ponderada de las mismas variables estructurales empleadas como predictores, una regresión lineal simple lo recupera de forma casi directa (R^2=0,919), mientras que XGBoost —un modelo de árboles, comparativamente ineficiente ante relaciones puramente lineales, más aún con una muestra de n=32— se queda por detrás (R^2=0,756). En consecuencia, los resultados de esta fase deben interpretarse no como una predicción de demanda real de pasajeros, sino como una estimación del nivel de servicio estructural esperado de cada ruta, empleando la brecha entre valor real y predicho como herramienta exploratoria de priorización, no como el error de un modelo predictivo en sentido estricto.

El segundo punto, la ausencia de un modelo baseline de referencia contra el que contextualizar el R^2 de 0.756, se ha resuelto incorporando dos baselines evaluados con la misma validación LOO-CV: una media global (predictor ingenuo) y una regresión lineal sobre las mismas ocho features. Los resultados se resumen en la siguiente tabla:

Modelo	**MAE (LOO-CV)**	**R^2 (LOO-CV)**
Media global (baseline)	14.99	-0.066
Regresión lineal (baseline)	3.44	0.919
XGBoost (modelo actual)	6.16	0.756

El resultado es revelador y obliga a matizar la lectura del modelo: la regresión lineal no solo supera al baseline de media global (como cabía esperar), sino que también supera al propio XGBoost, tanto en error (MAE 3.44 frente a 6.16) como en ajuste (R^2 0.919 frente a 0.756). Esto tiene una explicación directa ligada al propio diseño del IDE: al ser una combinación lineal de variables normalizadas, un modelo lineal está mejor capacitado para recuperarlo que un modelo de árboles con no linealidades que aquí no existen en el target. En otras palabras, gran parte de lo que XGBoost "aprende" no es una relación compleja genuina, sino una aproximación menos eficiente a una fórmula lineal ya conocida parcialmente por el propio diseño de las features. Este hallazgo refuerza, con evidencia empírica y no solo argumental, la lectura de leakage parcial planteada en el punto anterior, y sugiere que un modelo lineal —más simple, más interpretable y con mejor desempeño aquí— sería una alternativa igual o más defendible que XGBoost para este caso de uso concreto.

El tercero es la ausencia de una justificación externa para los pesos empleados en la construcción del IDE, que constituyen una aproximación experta del autor y no una calibración basada en datos reales de viajeros. Se recoge como línea de mejora la realización de un análisis de sensibilidad que muestre cómo varía la brecha resultante ante cambios razonables en estos pesos, reforzando la robustez de las conclusiones frente a esta elección de diseño.

8. Fase 3 — Asistente conversacional

El objetivo de esta fase es ofrecer una interfaz en lenguaje natural que permita a un usuario sin conocimientos técnicos consultar los datos generados por las fases anteriores del proyecto, sin necesidad de interactuar directamente con los CSVs, el notebook o los dashboards.

Arquitectura

El asistente se construye sobre LangChain y el modelo GPT-4o-mini, y sigue un flujo de dos pasos. En el primer paso, el modelo recibe la pregunta del usuario junto con un prompt que describe el esquema de los cuatro DataFrames disponibles (df_rutas, df_cobertura, df_clusters, df_paradas), y genera código Python capaz de responder a esa pregunta operando sobre esos DataFrames. Ese código se ejecuta con exec() sobre los datos reales, capturando la salida mediante redirect_stdout. En el segundo paso, el modelo recibe el resultado bruto de la ejecución y redacta una respuesta en español, en lenguaje natural, dirigida al usuario final. Esta separación en dos pasos permite que el sistema combine la precisión de una consulta programática sobre los datos reales con la naturalidad de una respuesta conversacional, evitando que el modelo "invente" cifras sin fundamento en los datos.

Aplicación web

El asistente se expone a través de una aplicación Streamlit (app_nbus.py) con layout ancho y una barra lateral con la identidad visual de NBUS: colores corporativos reales (rojo #DA291C, negro #211915) y un logo servido como fichero estático, para lo cual fue necesario habilitar enableStaticServing = true en .streamlit/config.toml. La imagen de la barra lateral se implementa mediante una etiqueta <img> HTML directa en lugar de st.image(), para evitar el botón de pantalla completa que Streamlit añade por defecto a las imágenes. Para no penalizar el tiempo de carga inicial de la aplicación, la librería langchain_openai se importa de forma diferida, dentro de la función get_llm(), en lugar de al inicio del script. La interfaz incluye además botones de preguntas frecuentes como acceso rápido a consultas habituales.

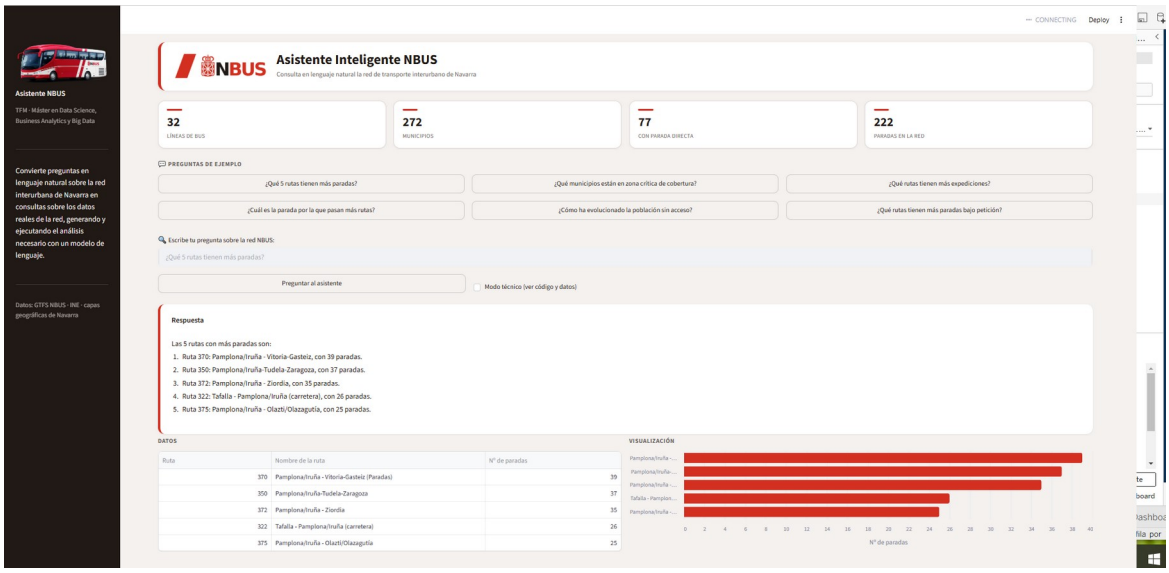


Figura 1. Interfaz del Asistente Inteligente NBUS, con preguntas de ejemplo y respuesta generada en lenguaje natural a partir de los datos reales.

Limitaciones y trabajo pendiente

La limitación más relevante de esta fase, desde el punto de vista de buenas prácticas de ingeniería, es que la ejecución del código generado por el modelo mediante exec() no restringe actualmente el espacio de nombres __builtins__, lo que en teoría permitiría que el modelo generase código con acceso a módulos como os u otras operaciones no deseadas sobre el sistema. La mitigación planteada es pasar un diccionario {'__builtins__': {}} o una whitelist explícita de funciones permitidas al entorno de ejecución de exec(), restringiendo el código generado estrictamente a operaciones sobre los DataFrames. Adicionalmente, el asistente no mantiene historial de conversación —cada pregunta se procesa de forma independiente— ni cachea respuestas a preguntas repetidas, lo que supone volver a invocar la API ante consultas idénticas. Ambos puntos se recogen como líneas de mejora futura.

9. Dashboards Tableau

Como capa complementaria al análisis cuantitativo y al asistente conversacional, se desarrollaron cuatro dashboards interactivos en Tableau, conectados directamente a los CSVs generados por el pipeline ETL, orientados a una exploración visual de la red por parte de perfiles tanto técnicos como no técnicos.

Dashboard	Contenido principal
Operaciones	Expediciones por ruta, frecuencia, actividad temporal
Cobertura	Cobertura por municipio, nivel de servicio
Población	Cruce servicio vs datos demográficos INE
Mapa / Rutas	Visualización geográfica de paradas y recorridos

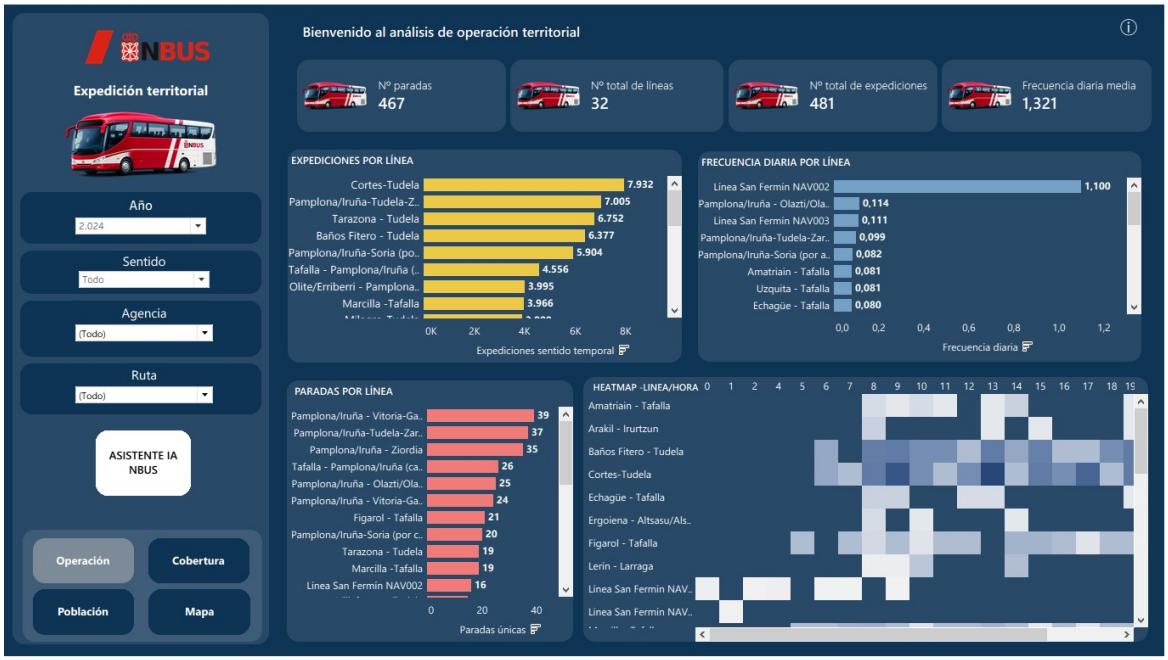


Figura 2. Dashboard de Operación: expediciones por línea, frecuencia diaria y heatmap línea/hora.

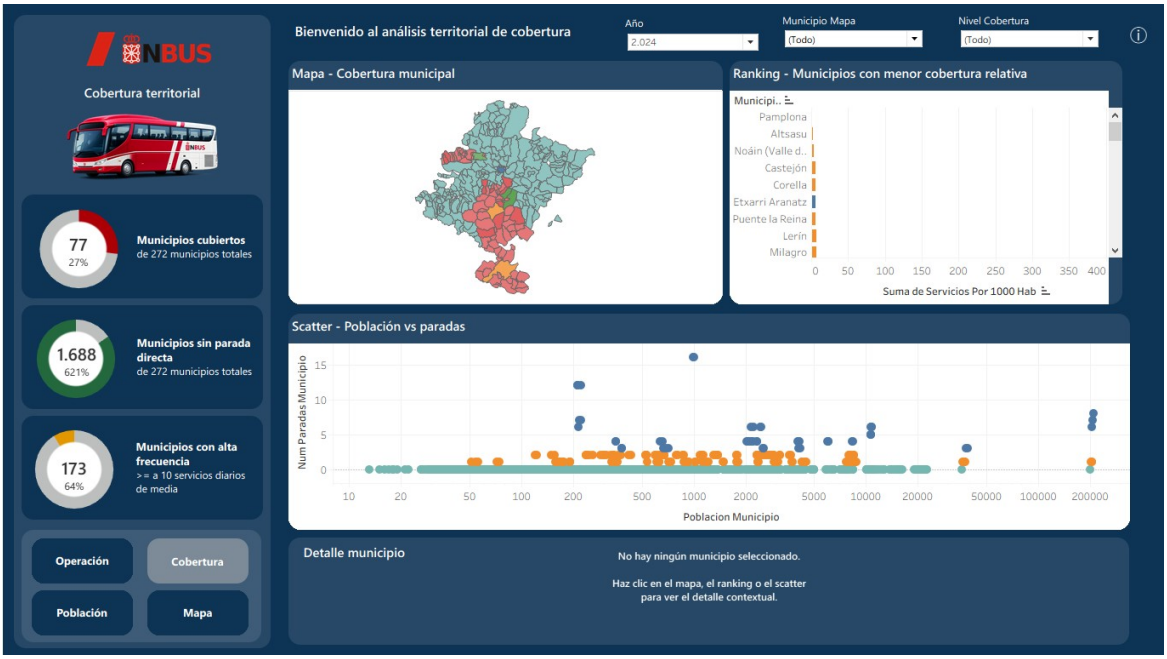


Figura 3. Dashboard de Cobertura: cobertura municipal, ranking de municipios con menor cobertura relativa y scatter población vs. paradas.

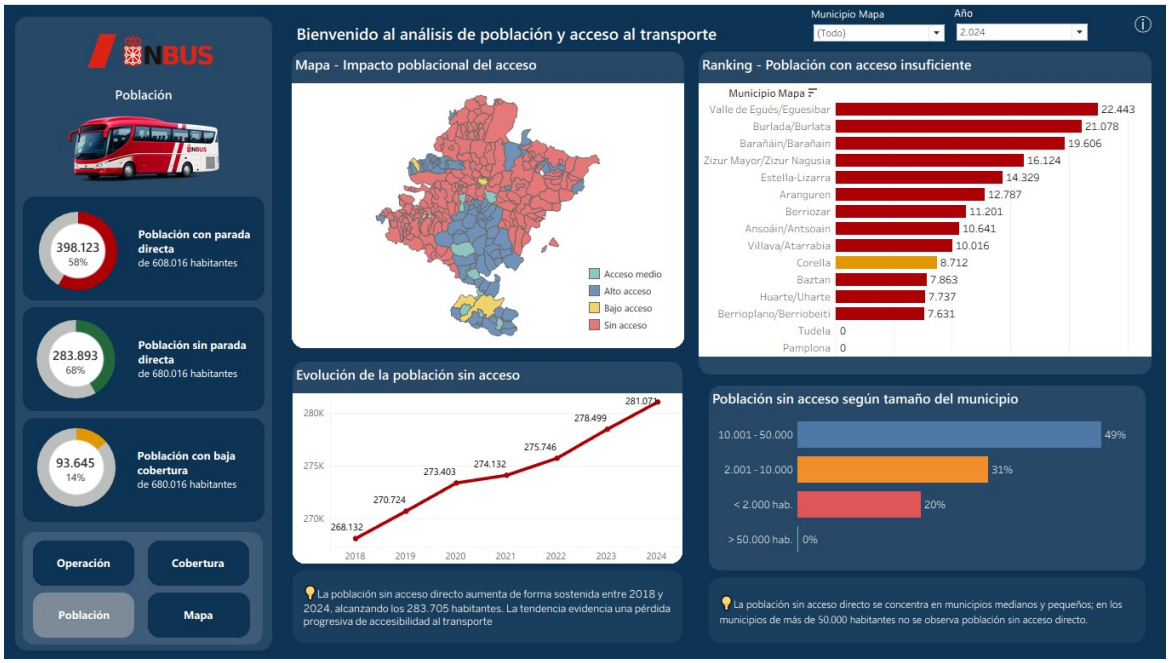


Figura 4. Dashboard de Población: impacto poblacional del acceso, evolución de la población sin acceso directo y ranking por municipio.

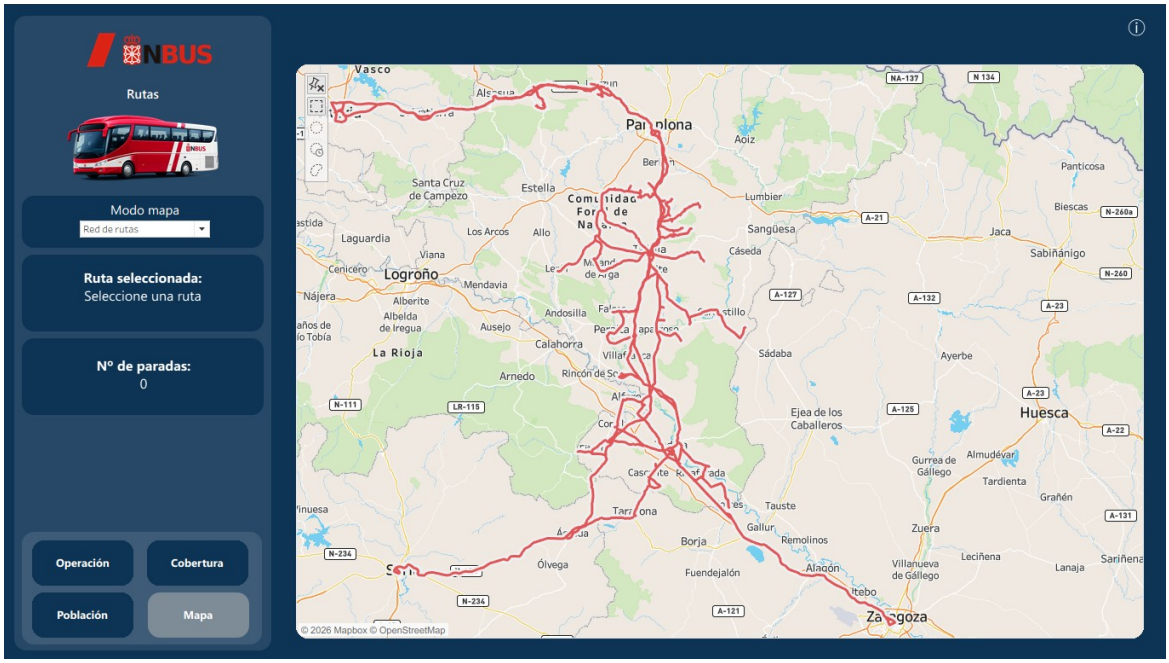


Figura 5. Dashboard de Mapa/Rutas: visualización geográfica de la red completa de rutas NBUS.

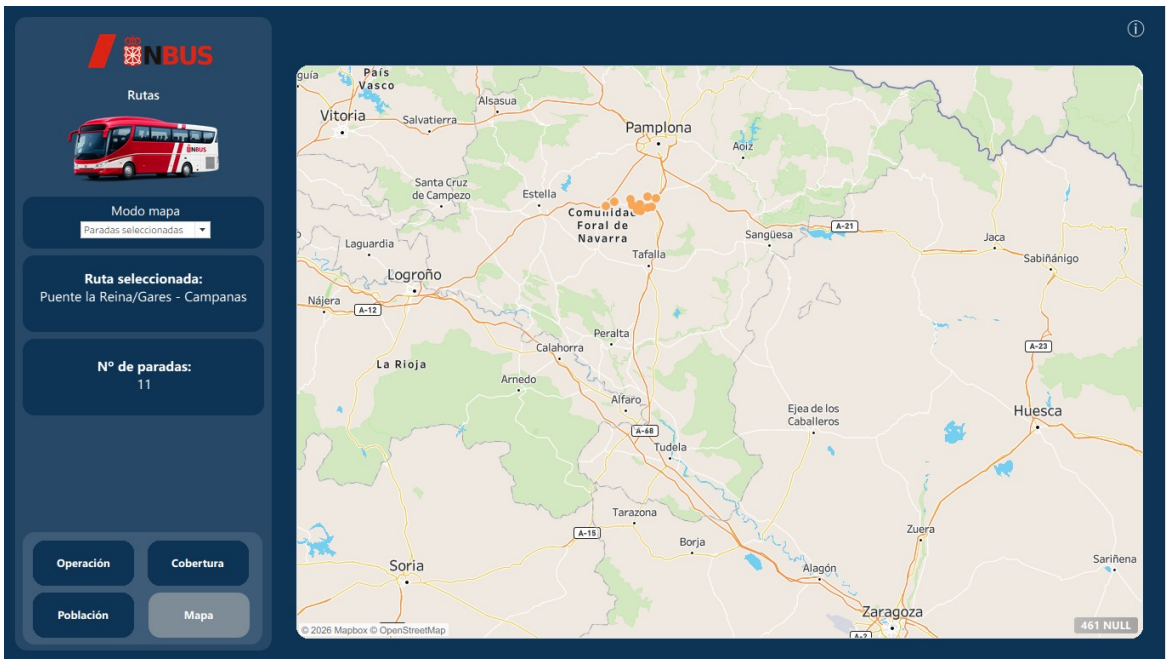


Figura 6. Dashboard de Mapa/Rutas filtrado por una ruta concreta (Puente la Reina/Gares - Campanas), mostrando sus paradas.

En el desarrollo de estos dashboards se tomaron dos decisiones técnicas relevantes. Por un lado, se migró de una fuente de datos única y muy amplia hacia múltiples CSVs específicos por dashboard, cada uno con solo las columnas necesarias para su propósito, lo que mejoró de forma notable el rendimiento de renderizado frente a la alternativa de una única fuente monolítica. Por otro lado, se emplearon expresiones de nivel de detalle (LOD) de tipo FIXED para el cálculo de KPIs que deben mantenerse constantes a nivel de ruta con independencia del contexto de filtrado aplicado por el usuario, evitando que estos indicadores se recalculasen incorrectamente al interactuar con los filtros. Además, se incorporaron parámetros para filtrar por dirección (ida/vuelta) y por año.

Estos dashboards no sustituyen al asistente conversacional, sino que lo complementan: mientras el asistente resuelve preguntas puntuales formuladas en lenguaje natural, los dashboards permiten una exploración visual y abierta de la red, más adecuada para el análisis comparativo entre rutas o municipios.

10. Resultados y discusión integrada

Los resultados de las tres fases analíticas, aunque generados de forma independiente, cobran su verdadero valor al leerse de manera conjunta. La segmentación de municipios (Fase 1) identifica dónde existen desequilibrios de cobertura desde una perspectiva territorial y demográfica; la estimación de demanda por ruta (Fase 2) identifica qué rutas concretas presentan una brecha entre servicio ofrecido y servicio esperado dado su perfil estructural; y el asistente conversacional (Fase 3) pone ambos resultados —junto con los datos operativos de base— al alcance de un usuario no técnico, sin necesidad de interpretar tablas ni gráficos por sí mismo.

Esta integración permite, por ejemplo, cruzar el cluster de un municipio con la brecha de demanda de las rutas que lo atraviesan, o preguntar directamente al asistente qué municipios con bajo nivel de cobertura son atravesados por rutas con demanda estimada superior a la ofrecida, una consulta que combinaría información de las tres capas y que sería costosa de responder manualmente cruzando los CSVs por separado. Los dashboards de Tableau, por su parte, ofrecen una vía de verificación visual de estos mismos patrones, útil para contrastar de forma independiente las conclusiones obtenidas por los modelos.

Dicho esto, la lectura conjunta de los resultados debe hacerse con las cautelas metodológicas expuestas en las secciones anteriores: la segmentación de municipios se apoya en un método del codo sin validación cuantitativa adicional (silhouette) y sobre una variable de población fuertemente asimétrica; y la estimación de demanda se basa en una variable objetivo construida, con un solapamiento parcial entre features y target confirmado empíricamente al comparar con los baselines. Esto no invalida el valor del proyecto como prototipo funcional y como demostración de un flujo end-to-end de analítica avanzada, pero sí acota el alcance de sus conclusiones: los resultados deben interpretarse como una primera aproximación exploratoria y priorizadora, no como una estimación definitiva y calibrada de la demanda real de la red.

En conjunto, el proyecto demuestra que es posible construir, a partir de datos abiertos GTFS y sin acceso a datos reales de viajeros, una herramienta analítica integral que combina segmentación, predicción e interacción en lenguaje natural sobre una red de transporte público real, sentando una base sólida sobre la que iterar con datos y validaciones adicionales.

11. Conclusiones y trabajo futuro

Este trabajo ha demostrado la viabilidad de construir, a partir de datos abiertos GTFS y sin necesidad de datos reales de viajeros, un sistema analítico end-to-end sobre una red de transporte público interurbano real, que integra ingeniería de datos, aprendizaje automático e inteligencia artificial generativa en un flujo coherente donde cada capa se apoya en las salidas de la anterior. El pipeline ETL desarrollado transforma ficheros GTFS en información limpia y reutilizable; el clustering de municipios ofrece una lectura territorial de la cobertura de servicio; el modelo de predicción de

demanda, pese a sus limitaciones, aporta un marco cuantitativo para detectar rutas potencialmente sub- o sobreservidas; y el asistente conversacional traslada todo este análisis a una interfaz accesible para usuarios no técnicos.

Entre las lecciones aprendidas destaca, en primer lugar, la importancia de examinar con rigor el diseño de variables construidas cuando no existe una etiqueta real disponible: la definición del IDE fue necesaria ante la ausencia de datos de viajeros, pero exige ser explícito sobre sus limitaciones en lugar de presentar sus métricas de rendimiento sin matices. En segundo lugar, la elección de técnicas de validación debe adaptarse al tamaño real de la muestra disponible, como ilustra el uso de leave-one-out ante solo 32 rutas. En tercer lugar, la integración de un asistente basado en LLM sobre datos reales introduce consideraciones de seguridad —en este caso, la ejecución de código generado por el modelo — que deben abordarse desde el diseño y no como una incorporación posterior.

Como líneas de trabajo futuro se identifican, de forma priorizada: aplicar una transformación logarítmica (log1p) sobre la variable de población municipal antes de escalar, y evaluar si la segmentación resultante cambia de forma sustancial respecto a la ya validada mediante silhouette score; explorar el uso de una regresión lineal como modelo principal de demanda en lugar de XGBoost, dado que supera a este último tanto en error como en ajuste; realizar un análisis de sensibilidad sobre los pesos del IDE; llevar a cabo un análisis de residuos del modelo de demanda para identificar qué rutas se predicen peor y por qué; incorporar valores SHAP para reforzar la interpretabilidad del modelo; restringir el entorno de ejecución del asistente conversacional mediante una whitelist de funciones permitidas, cerrando el riesgo de seguridad identificado en el exec(); y cruzar la clasificación de municipios "Sin servicio" con los datos del servicio de Transporte a la Demanda de NBUS (taxi concertado por reserva telefónica en municipios de baja densidad), no incluido en el GTFS de partida, para distinguir entre municipios sin ningún tipo de transporte público y municipios cubiertos por esta modalidad alternativa no capturada en los datos actuales.

En conjunto, estas líneas de mejora no cuestionan la validez del prototipo desarrollado, sino que marcan el camino natural para convertirlo en una herramienta más robusta y defendible desde el punto de vista metodológico, manteniendo la arquitectura end-to-end como su principal fortaleza.

12. Bibliografía y referencias

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. En Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability (Vol. 1, pp. 281–297). University of California Press.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, 20, 53–65.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., &

Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Google. (2024). GTFS static overview [Documentación]. Google for Developers. https://developers.google.com/transit/gtfs

Chase, H. (2022). LangChain [Software]. GitHub. https://github.com/langchain-ai/langchain

Gobierno de Navarra. Datos abiertos de transporte — Red NBUS. Portal de datos abiertos de Navarra.

NBUS — Gobierno de Navarra. Transporte a la Demanda. https://nbus.navarra.es/es/transporte-a-demanda

Instituto Nacional de Estadística. (2024). Población por municipios y sexo (tabla 2884). https://www.ine.es/jaxiT3/Tabla.htm?t=2884

OpenAI. GPT-4o mini Model Documentation. https://platform.openai.com/docs/models

Streamlit Inc. Streamlit Documentation. https://docs.streamlit.io/

Tableau Software. Tableau Desktop — LOD Expressions Documentation. https://help.tableau.com/

Mahmoudi, R., Saidi, S., & Wirasinghe, S. C. (2024). A critical review of analytical approaches in public bus transit network design and operations planning with focus on emerging technologies and sustainability. Journal of Public Transportation, 26, 100100. https://www.sciencedirect.com/science/article/pii/S1077291X24000201

Rocco di Torrepadula, F., Napolitano, E. V., Di Martino, S., & Mazzocca, N. (2024). Machine learning for public transportation demand prediction: A systematic literature review. Engineering Applications of Artificial Intelligence, 137(Part B), 109166. https://www.sciencedirect.com/science/article/abs/pii/S0952197624013241

Vansteenwegen, P., Melis, L., Aktaş, D., Montenegro, B. D. G., Vieira, F. S., & Sörensen, K. (2022). A survey on demand-responsive public bus systems. Transportation Research Part C: Emerging Technologies, 137, 103573. https://www.sciencedirect.com/science/article/abs/pii/S0968090X22000195